

RADEETA Management System

Resume court, professionnel et base uniquement sur le code ouvert dans VS Code.

Date de generation 10/05/2026 17:40	Sources analysees frontend/ et backend/
Confiance Analyse du code + verification des routes Laravel + 15 tests backend passes.	Format Fiche de revision pour presentation orale rapide.

Je n'ai pas invente de fonctionnalites. Quand un point semble incomplet ou pas totalement branche dans le code actuel, il est marque 'a verifier'.

1. Presentation generale

Point	Resume
Nom du projet	Depot : RADEETA-Management-System
Nom visible dans l'interface	SRM-FM
Objectif principal	Centraliser la gestion des reclamations clients, des secteurs, des compteurs, des interventions techniques et des rapports.
Probleme resolu	Eviter une gestion dispersee des incidents terrain et faciliter le suivi par role dans une seule application web.

Le projet sert a suivre les operations d'une regie technique : zones geographiques, abonnees, compteurs, pannes, reparations, interventions, notifications et rapports.

Le nom du depot est 'RADEETA-Management-System', mais l'interface affiche surtout 'SRM-FM'. Cela montre probablement une evolution du projet ou un changement de contexte : a verifier si ton professeur attend un seul nom officiel.

Role	Utilite
directeur	Acces global, administration, utilisateurs, parametres, journaux.
responsable	Pilotage principal du systeme, CRUD metier, dashboard, rapports.

Role	Utilite
manager	Suivi operationnel, pannes, interventions, rapports, compteurs.
technician	Travail terrain sur les pannes assignees, reparations, interventions, taches.
viewer	Lecture seule sur plusieurs modules metier.
developer	Role technique present dans le code, bloque en production.

2. Technologies utilisees

Categorie	Technologies observees
Frontend	React 19, Vite 8, Tailwind CSS, React Router 7
Backend	Laravel 11, PHP 8.2
Base de donnees	MySQL observe dans backend/.env + Eloquent ORM
Authentification	Laravel Sanctum + token Bearer + middleware de role
Bibliotheques importantes	Axios, Recharts, Leaflet/react-leaflet, i18next, lucide-react, framer-motion, DomPDF, Laravel Excel
Tests / qualite	PHPUnit cote backend, ESLint cote frontend

- Le frontend est une SPA React compilee par Vite.
- Le backend expose une API REST Laravel sous /api.
- La communication se fait avec Axios, en JSON, via un token Bearer.
- Les exports de documents passent par DomPDF et Laravel Excel.

3. Architecture du projet

```
RADEETA-Management-System/
|-- frontend/
|   |-- src/
|   |   |-- api/
|   |   |-- components/
|   |   |-- context/
|   |   |-- dashboards/
|   |   |-- layouts/
|   |   |-- pages/
|   |   |-- routes/
|   |   |-- utils/
|   |-- package.json
|-- backend/
|   |-- app/
|   |   |-- Enums/
|   |   |-- Http/
|   |   |-- Controllers/
```

```

|-- Middleware/
|-- Requests/
|-- Resources/
|-- Models/
|-- Notifications/
|-- Policies/
|-- Services/
|-- Support/
|-- database/
|-- migrations/
|-- factories/
|-- seeders/
|-- routes/api.php
|-- tests/

```

Element	Role
Frontend	Affiche les pages, les dashboards, les formulaires, la carte et les tableaux.
Backend	Valide les donnees, applique les roles et gere la logique metier.
Communication	Axios appelle l'API Laravel. L'URL vient de VITE_API_BASE_URL, sinon http://localhost:8000/api.

Le projet suit une separation claire : React pour l'interface et Laravel pour la logique, la securite et la base de donnees.

4. Base de donnees

La base observee dans backend/.env est MySQL. Les donnees sont geres avec Eloquent et les migrations Laravel.

Table	Role	Relations importantes
users	Comptes de connexion et roles	1 utilisateur peut etre technicien assigne, createur de releve ou auteur de journal.
secteurs	Zones geographiques avec emplacement, agence et coordonnees	1 secteur contient plusieurs compteurs.
clients	Abonnes avec police, CIN, telephone, adresse, type d'abonnement	1 client peut avoir plusieurs compteurs.
compteurs	Compteurs lies a un client et un secteur	1 compteur peut avoir plusieurs pannes et releves.
pannes	Reclamations / anomalies sur les compteurs	1 panne appartient a 1 compteur et peut etre assignee a 1 technicien.
reparations	Actions de reparation liees a une panne	1 reparation appartient a 1 panne et 1 technicien.
interventions	Rapports techniques plus detailles	Lies a la panne, au client, au compteur et au technicien.

Table	Role	Relations importantes
releves	Lectures d'index de compteur	Calcul automatique de la consommation.
settings	Parametres applicatifs	Reserve au directeur.
activity_logs	Historique d'actions	Trace les connexions et modifications importantes.
notifications	Notifications Laravel en base	Utilisees dans la cloche du frontend.
personal_access_tokens	Tokens Sanctum	Gere l'authentification API.

- 1 secteur -> plusieurs compteurs.
- 1 client -> plusieurs compteurs.
- 1 compteur -> plusieurs pannes et plusieurs releves.
- 1 panne -> plusieurs reparations et plusieurs interventions.
- 1 technicien -> plusieurs pannes assignees, reparations et interventions.
- Plusieurs tables utilisent la suppression logique (soft deletes).

5. Fonctionnalites principales

Fonctionnalite	Ce que fait le projet
Authentification	Connexion par identifiant ou email, mot de passe, token Sanctum et deconnexion.
Dashboards	Statistiques, graphiques Recharts, carte des pannes et indicateurs par role.
CRUD metier	Clients, compteurs, secteurs, pannes, reparations, interventions.
Gestion des utilisateurs	Creation, mise a jour, attribution des roles, module administration.
Affectation des pannes	Un technicien ne voit que les pannes qui lui sont assignees.
Reparations	Une reparation valide ferme automatiquement la panne associee.
Interventions	Suivi des travaux, priorite, materiaux, dates et statut.
Notifications	Cloche frontend avec marquage comme lu et rechargement regulier.
Rapports	Export PDF des pannes et export Excel des clients/compteurs.
Cartographie	Carte Leaflet des secteurs et incidents selon les coordonnees GPS.
Releves	Module backend et page frontend presentes, consommation calculee automatiquement.

Pages ou modules presents dans le code mais a verifier dans l'experience finale : Releves.jsx et Notifications.jsx existent, mais ne sont pas branches comme pages principales dans App.jsx.

6. Routes importantes

Routes frontend

Route	Role principal
/login	Page de connexion
/access-denied	Page de refus d'accès
/	Redirection automatique selon le rôle
/administration	Administration du directeur
/admin/*	Espace responsable : dashboard, clients, compteurs, secteurs, pannes, réparations, interventions, rapports
/manager/*	Espace manager : dashboard, pannes, réparations, interventions, rapports
/technician/*	Espace technicien : dashboard, tâches, pannes, réparations, interventions
/operator/*	Alias redirige vers /technician/*
/viewer/*	Espace lecture : dashboard, clients, compteurs, secteurs, pannes, réparations, interventions, rapports

Routes backend / API

Endpoint	Utilité
POST /api/login	Authentifier l'utilisateur et créer le token
GET /api/me, POST /api/logout	Lire le profil courant et fermer la session
GET /api/dashboard/stats	Charger les indicateurs du dashboard
GET/POST/PUT/DELETE /api/clients	CRUD des clients
GET/POST/PUT/DELETE /api/compteurs	CRUD des compteurs
GET/POST/PUT/DELETE /api/secteurs	CRUD des secteurs
GET/POST/PUT/DELETE /api/pannes	Gestion des réclamations et affectation
GET/POST/PUT/DELETE /api/reparations	Gestion des réparations
GET/POST/PUT /api/interventions	Gestion des interventions
GET/POST/PUT /api/relevés	Gestion des relevés

Endpoint	Utilite
GET/POST /api/notifications*	Lister et marquer les notifications comme lues
GET/PUT /api/settings	Lire et modifier les parametres
GET /api/logs	Consulter le journal d'activite
GET/POST/PUT /api/users*	Gestion des utilisateurs et liste des techniciens
GET /api/reports/pannes/pdf	Exporter le rapport PDF des pannes
GET /api/reports/clients/excel	Exporter le rapport Excel clients/compteurs

A verifier : la route frontend /viewer/reports existe, mais les exports backend de rapports sont limites a des roles plus eleves dans routes/api.php.

7. Exemples de code importants

Exemple frontend : protection des pages

```
if (!isAuthenticated || !isKnownRole(role)) {
    return <Navigate to="/login" replace state={{ from: location }} />;
}

if (roles && !hasRole(role, roles)) {
    return <Navigate to="/access-denied" replace />;
}

return <Outlet />;
```

Explication simple : si l'utilisateur n'est pas connecte, il retourne vers /login. Si son role n'est pas autorise, il va vers /access-denied.

Exemple backend : une reparation ferme la panne

```
$reparation = DB::transaction(function () use ($validated): Reparation {
    $reparation = Reparation::create($validated);
    $reparation->panne()->update([
        'status' => PanneStatus::Resolved->value,
    ]);
    return $reparation;
});
```

Explication simple : la creation de la reparation et la mise a jour du statut de la panne se font ensemble dans une transaction.

Exemple modele : calcul automatique de consommation

```
protected static function booted(): void
{
    static::saving(function (Releve $releve): void {
        $releve->consommation = max(
```

```

        0,
        (float) $releve->nouvel_index - (float) $releve->ancien_index
    );
});
}

```

Explication simple : la consommation n'est pas saisie a la main. Elle est calculee automatiquement dans le modele Releve.

Exemple migration : table interventions

```

Schema::create('interventions', function (Blueprint $table): void {
    $table->id();
    $table->string('intervention_number')->unique();
    $table->foreignId('panne_id')->nullable()->constrained('pannes')->onDelete('nullOnDelete');
    $table->foreignId('client_id')->nullable()->constrained('clients')->onDelete('nullOnDelete');
    $table->foreignId('meter_id')->nullable()->constrained('compteurs')->onDelete('nullOnDelete');
    $table->foreignId('technician_id')->constrained('users')->restrictOnDelete();
    $table->enum('priority', ['low', 'normal', 'high', 'urgent'])->default('normal');
    $table->enum('status', ['en_attente', 'en_cours', 'terminee', 'annulee'])->default('en_attente');
});

```

Explication simple : cette table relie une intervention a une panne, un client, un compteur et un technicien, avec un niveau de priorite et un statut.

8. Comment expliquer le projet au professeur

Resume oral de 2 minutes

Ce projet est une application web de gestion technique et administrative. Le frontend React sert a afficher les tableaux, les dashboards, la carte et les formulaires, tandis que le backend Laravel expose une API securisee par roles. Le coeur metier repose sur la relation secteurs -> clients -> compteurs -> pannes, puis sur les reparations, interventions et releves. Un point fort du projet est que les techniciens ne voient que leurs taches assignees, alors que les responsables et managers ont une vue de pilotage plus large. Enfin, le projet propose aussi des notifications et des exports PDF/Excel, ce qui le rend proche d'un vrai outil professionnel.

Points forts du projet

- Architecture propre frontend / backend.
- Securite par roles assez bien pensee.
- Projet proche d'un vrai besoin metier terrain.
- Presence de dashboards, carte, notifications et exports.
- Base de donnees coherente avec le cycle complet client -> compteur -> panne -> intervention.
- 15 tests backend ont passe pendant la verification.

Questions possibles du professeur

Question	Reponse simple
Pourquoi avoir separe frontend et backend ?	Pour avoir une interface React fluide et une API Laravel claire, securisee et reutilisable.

Question	Reponse simple
Comment le projet est-il securise ?	Par la connexion Sanctum, le token Bearer, les routes protegees et les roles verifies aussi bien dans React que dans Laravel.
Comment limiter un technicien a ses propres donnees ?	Le backend filtre ses pannes, reparations, releves et interventions avec des regles metier dediees.
Que se passe-t-il quand une reparation est enregistree ?	La panne associee passe automatiquement au statut resolu.
Pourquoi avoir une carte ?	Pour localiser rapidement les secteurs et visualiser les incidents sur le terrain.
Y a-t-il des evolutions encore en cours ?	Oui : certaines pages existent dans le code mais leur exposition finale dans le routage ou le menu reste a verifier.

9. Points a verifier

- Nom officiel a utiliser a l'oral : RADEETA ou SRM-FM.
- Page Releves presente dans le code frontend et endpoint backend actif, mais pas routee dans App.jsx au moment de l'analyse.
- Page Notifications presente dans le code, mais l'interface utilise surtout la cloche NotificationBell dans le topbar.
- Le viewer a une route frontend vers les rapports, mais les exports API semblent restreints a d'autres roles.
- Le menu Sidebar n'affiche pas toutes les routes viewer pourtant presentes dans App.jsx.

Conseil pratique : si ton professeur demande les limites du projet, tu peux dire qu'il est fonctionnel et bien structure, mais que certaines parties semblent encore en cours d'alignement entre les pages, le menu et les autorisations.

Document genere depuis le code source disponible le 10/05/2026 17:40.